

Раздел 1. Алгоритмы и структуры данных

Содержание

1. Сортировка слиянием (mergesort). Определение числа инверсий в перестановке за $O(n \log n)$.
2. Быстрая сортировка (quicksort). Поиск k-й порядковой статистики за линейное в среднем время.
3. Двоичная куча. Пирамидальная сортировка. Построение кучи за линейное время.
4. Хеш-таблицы. Методы разрешения коллизий: метод цепочек, открытая адресация. Универсальное семейство хеш-функций.
5. Амортизированная сложность. Динамический массив и оценка сложности его операций.
6. Динамическое программирование: одномерное, двумерное.
7. Самобалансирующиеся деревья поиска (рассказать любое на выбор). Реализация и оценка сложности операций вставки, удаления, поиска.
8. Графы. Представление графов в памяти. Обход в ширину. Обход в глубину. Топологическая сортировка.
9. Задача поиска кратчайших путей в графе. Алгоритм Дейкстры. Алгоритм Форда–Беллмана.
10. Задача построения минимального остовного дерева. Алгоритм Крускала. Алгоритм Прима.
11. Задача о максимальном потоке в графе. Теорема Форда–Фалкерсона. Алгоритм Эдмондса–Карпа.
12. Дерево отрезков. Ограничения на поддерживаемую операцию. Запросы модификации на отрезке.
13. Быстрое преобразование Фурье. Применение для умножения длинных чисел и многочленов.
14. Нахождение обратного элемента по модулю. Быстрое возведение в степень.
15. Задача поиска подстроки в строке. Алгоритм Рабина–Карпа. Алгоритм Кнута–Морриса–Пратта.
16. Бор. Алгоритм Ахо–Корасик.

1. Сортировка слиянием (mergesort). Определение числа инверсий в перестановке за $O(n \log n)$.

Процедура Merge

Задача: Даны два отсортированных массива $a_1 \leq a_2 \leq \dots \leq a_n$ и $b_1 \leq b_2 \leq \dots \leq b_m$. Надо их слить в один отсортированный массив за $O(n + m)$ времени и доппамяти.

Алгоритм:

1. Заведём указатели на первые элементы массивов i и j .
2. По индексу $i + j$ выпишем меньший из a_i и b_j и сдвинем соответствующий указатель на один вперед.
3. Так делаем, пока $i < n$ и $j < m$.
4. Оставшийся хвост одного из массивов дописываем в конец.

MergeSort

Задача: Дан массив $a_1, \dots, a_n$. Необходимо отсортировать его за $O(n \log n)$ времени и $O(n)$ доппамяти.

Алгоритм:

1. Разделим массив на две равные части.
2. Каждую из них рекурсивно отсортируем.
3. Результаты сольём в один отсортированный массив.

Подсчёт числа инверсий

def Пара $i < j$ называется *инверсией*, если $a_i > a_j$.

Задача: Дан массив $a_1, \dots, a_n$. Необходимо найти число инверсий за $O(n \log n)$ времени и $O(n)$ доппамяти.

Решение:

1. Если при слиянии элемент левой части больше элемента правой части, то значит это и есть инверсия.
2. Пусть мы сравниваем в сортировке слиянием $l[i]$ и $r[j]$, тогда если $r[j] < l[i]$, то $r[j] < l[i] < l[i + 1] < \dots < l[N]$, то есть число инверсий это $N - i$ для $r[j]$.
3. Тогда для одного слияния итоговое число инверсий равно сумме по j таких значений.
4. Итоговый ответ равен сумме по всем слияниям.

2. Быстрая сортировка (quicksort). Поиск k-й порядковой статистики за линейное в среднем время.

QuickSort

Цель: Отсортировать массив $a_1, \dots, a_n$ за $O(n \log n)$ времени в среднем.

Алгоритм:

1. Определённым образом выбрать опорный элемент `pivot`.
2. Произвести процедуру `partition`: все, что меньше `pivot` левее него, а все, что больше – правее.
3. Вызвать рекурсивно от правой и левой половин.

th Среднее время работы быстрой сортировки массива $a_1, a_2, \dots, a_N$ при случайном выборе опорного элемента составит $O(N \log N)$.

□

Будем считать, что в массиве нет повторяющихся элементов. Перенумеруем их таким образом, что b_i – i -ый по возрастанию элемент.

Введём случайную величину – число сравнений: $N_{i1 j>i}^{i,j}$, где i,j – бинарная случайная величина, равная единице, если b_i сравнивался с b_j .

Пусть p – опорный элемент, тогда впоследствии не будут сравниваться элементы, попавшие в разные стороны от p . При этом b_i сравнивается с b_j только если один из них был выбран опорным элементом. Вероятность, что кто-то был выбран опорным элементом равна 2^{j-i+1} (всего промежутке $j - i + 1$ элемент). Откуда:

$$\mathbb{E} N_{i1 j>i}^{i,j} = \sum_{j>i} \mathbb{E} N_{i1 j>i}^{i,j} = \sum_{j>i} 2^{j-i+1} = \sum_{j>i} 2^{N-i-1} = \sum_{j>i} O(2^{N-i-1}) = O(N \log N)$$

из факта $\sum_{i=1}^N O(\log N) = O(N \log N)$

Поиск k -ой порядковой статистики

def -ая порядковая статистика массива — такой элемент, что после сортировки массива он окажется на k -ом месте.

Алгоритм:

1. Выбрать случай опорный элемент.
2. Провести разбиение.
3. Если индекс опорного элемента i окажется больше данного k , то ищем слева k -ую порядковую, в случае равенства завершаемся, в противном случае – $(i - 1)$ -ую порядковую.

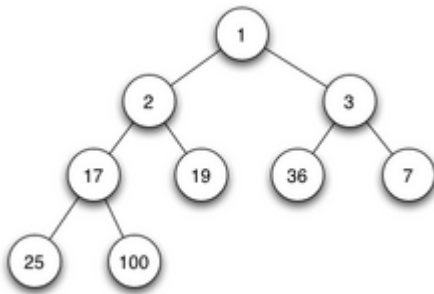
th (б/д) Время работы этого алгоритма линейно в среднем.

3. Двоичная куча. Пирамидальная сортировка. Построение кучи за линейное время.

Бинарная куча

def Куча – структура данных, которая позволяет добавлять в себя элементы и получать или извлекать минимум за логарифмическое время.

def Бинарная куча — структура данных в виде полного бинарного дерева, для которого верно свойство кучи: все дети больше родителя.



Просеивания: Процедуры для восстановления свойств кучи при условии, что их нарушает один элемент.

- `siftUp` (просеивание элемента вверх) – процедура, поднимающая элемент как можно выше, пока не выполняется свойство кучи.
- `siftDown` (просеивание элемента вниз) – процедура, погружающая элемент как можно ниже, пока не выполняется свойство кучи.

Операции:

- `getMin` – вернуть элемент, находящийся в корне.
- `insert` – подвесить элемент в самое левое свободное место на нижнем уровне и просеять вверх.
- `extractMin` – поменять местами корневой элемент и самый правый лист, лист отрезать, корень просеять вниз.

Бинарная куча может быть представлена как массив. В таком случае:

- Дети узла с индексом i имеют индексы $2i + 1$, $2i + 2$
- Родитель узла с индексом i имеет индекс $i - 1$

Построение за линию на массиве (Heapify)

def *Heapify* – функция, строящая кучу на заданном массиве. То есть надо переупорядочить элементы в массиве так, чтобы по индексам на всех парах родитель-сын было соблюдено свойство кучи.

Алгоритм: Будем просеивать вниз элементы, начиная с $N/2$ -ого до нулевого.

Корректность: Вытекает из корректности `siftUp` при корректных поддеревьях.

Время работы (б/д): $O(N)$

Сортировка кучей (HeapSort)

Цель: Отсортировать массив за $O(N \log N)$ времени и $O(1)$ доппамяти.

Алгоритм:

- Выполнить `heapify`.
 - `x = extractMin()`, записываем его в конец массива.
 - Повторяем $N - 1$ раз.
 - Развернуть массив.
-

4. Хеш-таблицы. Методы разрешения коллизий: метод цепочек, открытая адресация. Универсальное семейство хеш-функций.

def *Хеш-таблица* – контейнер, способный делать вставку, удаление и поиск в среднем за $O(1)$.

def Пусть S — множество рассматриваемых объектов, тогда $h: S \rightarrow \{0, 1, \dots, m-1\}$ называется *хеш-функцией*.

def Элементы $x, y \in S$, где $h(x) = h(y)$ образуют коллизию, если $x \neq y$.

Хеш-таблица на цепочках

def *Бакетом* (bucket) называют нечто, хранящее все элементы, образующие коллизию.

Создадим массив некоторой длины l , где будем хранить по индексу цепочку из данных. Поиск в цепочке работает за линейное время, поэтому мы хотим уменьшить их максимальную длину.

Рассмотрим величину q – длина цепочки, отвечающей ключу q . Пусть таблица построена для различных ключей $1, \dots, n$, тогда: $q_{ni1} I[(q) (i)]$

$$q_{ni1} I[(q) (i)]_{ni1} [(q) (i)]$$

$$((y)) 1, y1, y$$

С учётом того, что ключи различны:

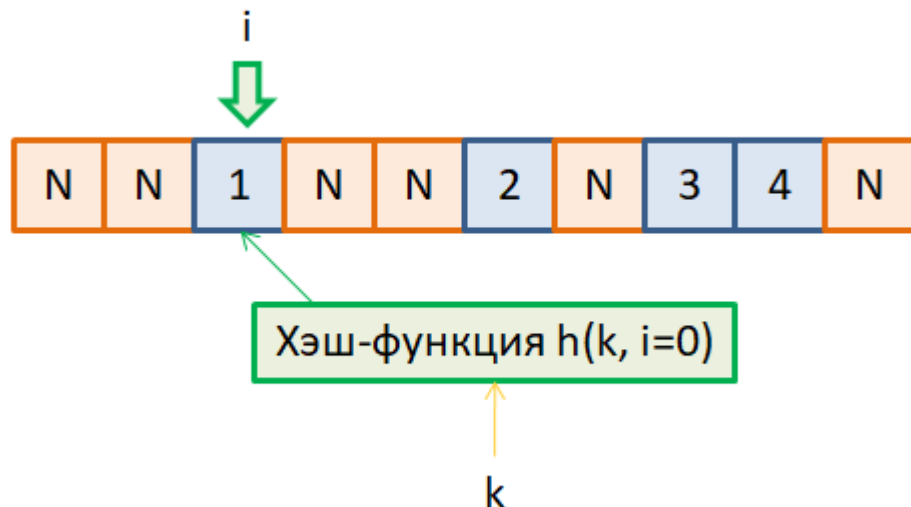
$$q_{ni1} [(q) (i)] \leq 1 + n-1 \leq 1 + n$$

def Коэффициентом загрузки (load factor) называют величину n .

Тогда заведём константу, т.ч. всегда $\leq n$. При соблюдении этого требования на load factor получим контейнер, выполняющий операции вставки, поиска и удаления за $O(1)$ в среднем.

Открытая адресация

Пусть вся хеш-таблица – один массив.



Пришёл, считаем $()$. Если $()$ занят – смотрим следующий.

def Пробой назовём функцию $(, i)$.

То есть это "более умный шаг". Я не хочу давать конкретики, поэтому примеры:

- *Линейное пробирование:* $(, i) () + ai + b$
- *Квадратичное пробирование:* $(, i) () + ai^2 + bi +$
- *Двойное хеширование:* $(, i) 1() + i \cdot 2()$

Операции в хеш-таблице с открытой адресацией:

- *Insert*: Считаем $(i, 0)$. В итоге вставим в (i, j) , где j – номер итерации прыжка, где нашли свободное место.
- *Lookup*: Идём от $(i, 0)$ до пустого, сравнивая в каждой ячейке с x . Если дошли до пустой, раньше, чем нашли x , то его не было.
- *Erase*: Просто удаление из ячейки может сломать "пути/цепочки поиска" других элементов. Введём новое состояние – *tmbstn*. *Insert* будет видеть томбу как пустую ячейку, и сможет поверх неё перезаписать новый элемент. *Lookup* же воспринимает томбу как элемент, и идёт дальше по пробе.

Поддерживаем load factor $\leq \alpha$. Если $\alpha > 0$, то рехеш+реаллокация:

$m \rightarrow 2m, () \rightarrow () \circ m \rightarrow () \rightarrow () \circ 2m$.

Время работы: $O(1)$ в среднем, вставка $O(1)$ в среднем.

Универсальное семейство хеш-функций

def – универсальное семейство хеш-функций. $y \rightarrow (y) \leq 1/m$

Пример: $a, b \rightarrow ((a + b) \circ m) \circ m$, где m – простое, $y \in \{0, 1, \dots, m-1\}$, $a, b \in \{0, 1, \dots, m-1\}$

st Данное семейство универсально.

□

$a + b \rightarrow ay + b \circ a \rightarrow ay \circ a$ взаимно просто $y \circ m$ т.к. $y, y \circ m$ – взаимно просто.

Коллизий на первом этапе нет.

$$a + b \rightarrow 1a + b \rightarrow \begin{pmatrix} 1 & y \\ 0 & 1 \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} \rightarrow \begin{pmatrix} a \\ b \end{pmatrix} \rightarrow \begin{pmatrix} 1 & y \\ 0 & 1 \end{pmatrix}^{-1} \begin{pmatrix} a \\ b \end{pmatrix} \rightarrow \begin{pmatrix} 1 & y \\ 0 & 1 \end{pmatrix}^{-1} \begin{pmatrix} a \\ b \end{pmatrix}$$

По $1, 2, \dots, y$ восстанавливаются a, b , то есть биекция $(y) \rightarrow (a, b)$

Зафиксируем (a, b) и посчитаем число $(y): (y) \rightarrow (a, b)$. Всего пар ключей $< m^2$, считаем число пар, дающих коллизию: фикс. (a, b) – способов. Число $y \rightarrow (y) \rightarrow (a, b) \rightarrow y \circ m \rightarrow m$

$$((y)) \rightarrow y((y)) \rightarrow m - 1$$

5. Амортизированная сложность. Динамический массив и оценка сложности его операций.

def Пусть имеется последовательность операций, каждая из которых выполняется за время t_i , тогда *амортизированная стоимость* операции $t = \sum_{i=1}^n t_i$

Амортизированная сложность обозначается $O(f(n))$. Для доказательства амортизированной сложности применяется метод монеток (бухучёт) или метод потенциалов.

Метод монеток:

- Допустим, что каждая операция стоит определенное количество монет i – учётная стоимость операции.
- Учётная стоимость не менее реальной. Оставшиеся монеты мы кладем в резерв, чтобы в будущем оплатить тяжёлую операцию.

Метод потенциалов:

def Потенциалом от структуры назовём такую функцию $\Phi()$, что:

- $\Phi(0) \geq 0$, где 0 – изначальное состояние структуры;
- $\Phi() \geq 0$ в любой момент времени.

def Пусть t_i – реальное время i -ой операции, тогда $t_i = t_i + \Phi(i) - \Phi(i-1)$ – учётное время.

st $\sum_{i=1}^n t_i = t$

Динамический массив

def *Динамически расширяющийся массив* (далее везде буду называть вектор) – массивоподобная структура, предлагающая следующий интерфейс:

- `push_back(n)` – добавить значение `n` в конец вектора.
- `pop()` – возвращает значение в конце вектора и удаляет его.
- `get(i)` – получить значение по индексу `i`

Реализация на массиве: у нас есть массив `T[] data`, его максимальная вместимость `cap = data.length` и нынешняя длина `size`.

- При добавлении элемента в конец перезаписываем его поверх `data[size]` и `size+=1`. При удалении с конца `size-=1`.
- Если добавляем в конец, но `size == cap`, то создаём новый массив `T[]` длины `cap*2`, и перезапишем в него `size` элементов из старого массива. Потом как обычно добавляем.
- Если удаляем с конца и вышло, что `size < cap / 4`, то создаём новый массив длины `cap/2`, и перезаписываем туда элементы.

Амортизационный анализ (методом монеток):

Рассмотрим на примере `push_back` в векторе, `pop` аналогично:

- При добавлении элемента платим 3 монетки – одну за запись в массив, и две в резерв.
- Инвариант: к следующей реаллокации у нас монет в резерве не меньше, чем элементов.
- При реаллокации платим `cap` монеток на перемещение элемента. Повторные реаллокации будут оплачены резервом с добавления `cap` значений в правую половину.

6. Динамическое программирование: одномерное, двумерное.

{WARNING: Я хз, что тут требуют}

def Динамическое программирование – подход к решению задач, где рассматривается сведение подзадачи к меньшей с оптимизированным перебором возможных вариантов.

Пять шагов для решения задач на ДП:

1. Что хранит в себе состояние динамики?
2. База.
3. Как пересчитывать?
4. Порядок пересчёта.
5. Где лежит ответ?

Одномерное ДП

В одномерных задачах ДП: – одномерный массив состояний, где i -ое состояние зависит от всех предыдущих, т.е. $f([i-1], \dots, [1], [0])$.

Примеры таких задач:

- **Числа Фибоначчи:** $[i] = [i-1] + [i-2]$, $[0] = 1$, $[1] = 1$
- **Задача о Кузнечике:** Дан массив $a_1, \dots, a_n$. Находясь изначально в $a_0 = 0$ необходимо допрыгать до a_n , максимизировав сумму a_i , в которых побывал кузнечик. Он может прыгать на 1 или 2 клетки вперёд.

Решение: Пусть $[i]$ – ответ для первых i клеток. База: $[0] = 0$, $[1] = a_1$. Переход $[i] = a_i + \max([i-1], [i-2])$. Цикл по i . Ответ в $[n]$.

- **Задача о Наибольшей возрастающей последовательности:** Дан массив $a_1, \dots, a_n$. Необходимо извлечь наибольшую по длине возрастающую подпоследовательность. То есть найти такие индексы $1 \leq i_1 < i_2 < \dots < i \leq n$, что $a_{i_j} < a_{i_{j+1}}$ и максимально.

Решение: Пусть $[i]$ – длина НВП, кончающейся в a_i . База $[0] = 1$. Шаг $a[i] = 1 + \max_{j < i, a_j < a_i} [j]$.

Цикл по i . Ответ: $ma()$.

Это можно сделать лучше, но мне сейчас лень это расписывать.

Двумерное ДП

В двумерных задачах ДП: – двумерный массив состояний, где (i, j) -ое состояние зависит от всех предыдущих, т.е. $f([i - 1][j], [i][j - 1], \dots)$.

Примеры:

Наибольшая общая подпоследовательность

Даны два массива $a_1, \dots, a_n$ и $b_1, \dots, b_m$. Необходимо извлечь наибольшую по длине общую подпоследовательность. То есть найти такие индексы $1 \leq i_1 < \dots < i \leq n$ и $1 \leq j_1 < \dots < j \leq m$, что $a_{i_l} = b_{j_l}$ и максимально.

Решение:

Пусть $[i][j]$ – ответ для $a[1..i], b[1..j]$.

База: $[0][0] = 0$.

Переход:

$$[i][j] = \begin{cases} ma([i - 1][j - 1]) + 1, & a_i = b_j \\ \max(ma([i - 1][j], [i][j - 1])), & \text{иначе} \end{cases}$$

Порядок пересчёта: `for i in range(2, n): for j in range(2, m):`

Ответ в $[n][m]$.

Рюкзак (0/1)

Даны два вектора $(a_1, \dots, a_n)$ и $(b_1, \dots, b_n)$, оба вектора из $\mathbb{N}$. Нужно построить битовый вектор $b \in \{0, 1\}^n$ такой, что

$$(a, b) \leq (a, b) \text{ и } (a, b) \text{ максимально}$$

Надо придумать решение за $O(nW)$.

Решение:

Пусть $[i]$ – ответ, если разрешено брать только первые i предметов, а ограничение по весу в рюкзаке.

База: $[0] = 0$

Переход:

$$[i] = \max([i-1], \text{i-й предмет берём} [i-1] - i + i, \text{i-й предмет берём})$$

Порядок пересчёта: `for i in range(1, n): for w in range(1, W)`

Ответ в $[n]$.

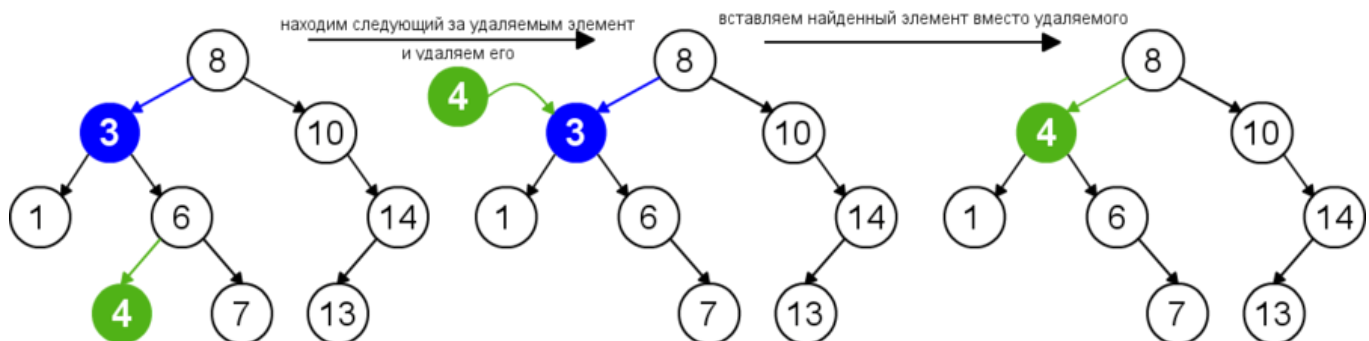
7. Самобалансирующиеся деревья поиска (рассказать любое на выбор). Реализация и оценка сложности операций вставки, удаления, поиска.

def Бинарным деревом поиска называют бинарное дерево, в котором в поддереве левее все элементы не больше элемента в данном узле, а в поддереве правее – строго больше.

Операции в дереве поиска: поиск, вставка, удаление за $O(h)$, где h – высота дерева.

В наивной реализации:

- **Поиск:** Начиная с корня, сравниваем со значением в узле. Если не совпало, идём в нужное поддерево.
- **Вставка:** Начиная с корня, сравниваем со значением в узле, идём в нужное поддерево, пока не придём в несуществующий узел. На его место подвешиваем новый.
- **Удаление:** Ищем узел. Если у него нет детей – просто удаляем, если 1 сын – подвешиваем на его место, если 2 сына, то схема с картинки:



Проблема простых деревьев – $O(N)$ высота в худшем случае (бамбук). Мы бы хотели высоту $O(\log N)$.

Тут можно разобрать AVL, Декартово Дерево, и Красно-Чёрное Дерево. В К/Ч слишком много поворотов, а для ДД нет доказательства в презах курса, поэтому проще всего разобраться с AVL.

AVL

def Дерево поиска является *AVL-деревом*, если для каждой вершины высота ее правого и левого поддеревьев различаются не более чем на единицу.

th Высота AVL-дерева равна $O(\log N)$.

□

Рассмотрим величину $F(n)$ – минимальное число вершин в AVL-дереве высоты n .

Заметим, что верно следующее соотношение: $F(n) = F(n-1) + F(n-2) + 1$

Докажем по индукции, что данную рекурренту можно выразить как $F(n) = F_{n+2} - 1$, где F_n – n -ое число Фибоначчи.

База: $F(0) = F_2 - 1, F(1) = F_3 - 1$

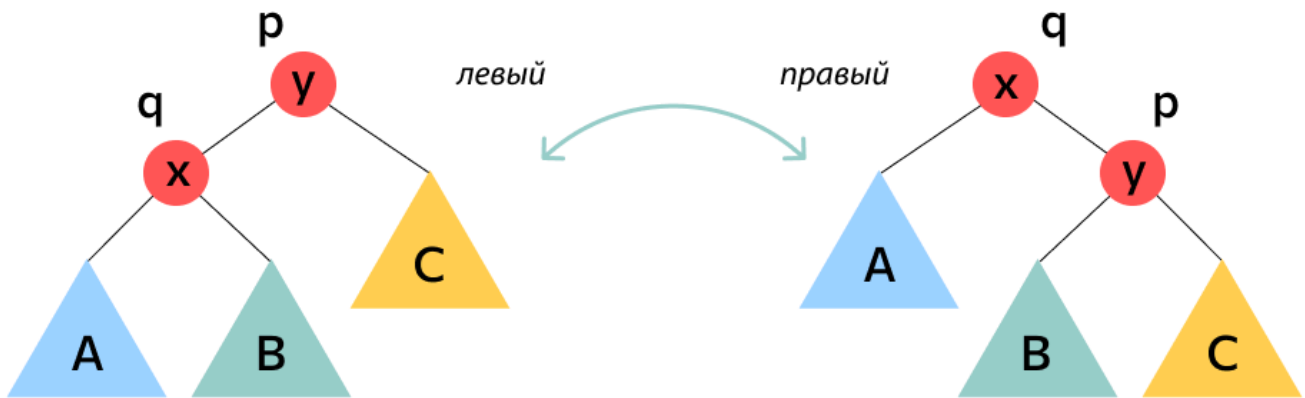
Переход: $F(n+1) = F(n) + F(n-1) + 1 = F_{n+2} - 1 + F_{n+1} - 1 + 1 = F_{n+3} - 1 = F_{(n+1)+2} - 1$

Таким образом, $N = F(n+2) - 1 \Rightarrow n = O(\log N)$

def *Балансировкой* AVL-дерева будем называть процесс, который из дерева поиска, в узле которого разность высот поддеревьев равна двум, переподвешивает детей и внуков так, чтобы выполнялось свойство AVL-дерева.

Балансировка в AVL-дереве представляет из себя применение одного из четырех видов поворотов вдоль пути от узла к корню.

Базовая операция – левые и правые повороты:

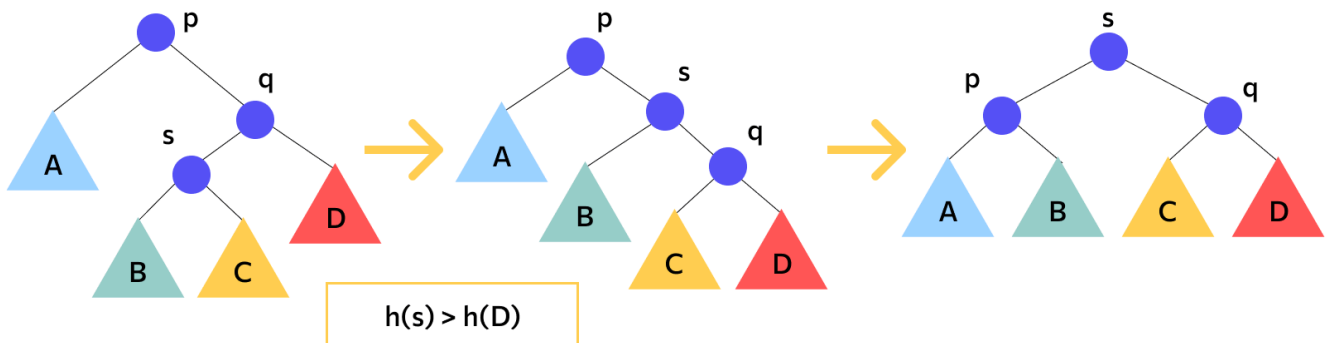


$$A < x < B < y < C$$

Из них собираются *малый левый поворот (L)*, *большой левый поворот (LR)*, *малый правый поворот (R)*, *большой правый поворот (RL)*:

Тип вращения	Иллюстрация	Когда используется	Расстановка балансов
Малое левое вращение		$diff[a] = -2$ и $diff[b] = -1$ или $diff[a] = -2$ и $diff[b] = 0$.	$diff[a] = 0$ и $diff[b] = 0$ $diff[a] = -1$ и $diff[b] = 1$
Большое левое вращение		$diff[a] = -2$, $diff[b] = 1$ и $diff[c] = 1$ или $diff[a] = -2$, $diff[b] = 1$ и $diff[c] = -1$ или $diff[a] = -2$, $diff[b] = 1$ и $diff[c] = 0$.	$diff[a] = 0$, $diff[b] = -1$ и $diff[c] = 0$ $diff[a] = 1$, $diff[b] = 0$ и $diff[c] = 0$ $diff[a] = 0$, $diff[b] = 0$ и $diff[c] = 0$

Более конкретно, LR поворот выглядит так:



Правые повороты определяются симметрично. Нужный поворот выбирается в зависимости от размеров поддеревьев. Каждый из них выполняется за $O(1)$

Тогда операции в AVL:

- Поиск не изменяет дерево, поэтому как обычно.
- Вставка и удаление меняют какое-то поддерево, достаточно рассмотреть самый нижний затронутый узел.

8. Графы. Представление графов в памяти. Обход в ширину. Обход в глубину. Топологическая сортировка.

def *Граф* – пара (V, E) , где V – множество вершин, E – мультимножество рёбер.

Рёбра в памяти представляются парой вершин, в неориентированных графах при наличии ребра (u, v) также есть и ребро (v, u) .

Есть следующие способы хранения графа:

- **Матрица смежности** – двумерный массив размера $n \times n$, где $I[i][j] = 1$, т.е. бинарная матрица, хранящая на i -ой строке j -ом столбце наличие ребра
- **Список смежности** – массив, где $adj[u]$ – список всех соседей вершины u .

Сравнение по сложности операций:

Операции	Список Смежности	Матрица Смежности
$(u, v) \in E$?	$O(1)$	$O(1)$
$(u, v) \notin E$	$O(1)$	$O(1)$
Получить список соседей	$O(1)$	$O(n)$
Потребляемая память	$O(n + E)$	$O(n^2)$

Depth-First Search

def *DFS* (depth first search) – обход в глубину, т.е. обход графа, в котором вершины посещаются как только обнаружены.

Псевдокод:

```
private static void dfs(ArrayList<Integer>[] graph, int v, int visited) {
    visited[v] = true;

    for (int u: graph[v]) {
        if (!visited[u]) {
            dfs(graph, u, visited);
        }
    }
}
```

```

    }

    // Вот здесь выход из вершины (см. топсорт)
}

```

Чёт в таком духе. Без рекурсии пишется со стеком.

Breadth-First Search

def *BFS* (breadth-first search) – обход в ширину, т.е. обход графа, в котором сначала посещаются соседи начальной вершины, затем их соседи и т.д.

Алгоритм:

- Заведём очередь вершин в неё положим стартовую s
- Пока очередь не пустая:
 1. Извлечём вершину из начала очереди, пометим как посещённую.
 2. Добавим все смежные не посещённые вершины в очередь.

Псевдокод:

```

private static void bfs(ArrayList<Integer>[] graph, int s) {
    bool[] visited = new bool[];
    ArrayDeque<Integer> queue = new ArrayDeque<Integer>();

    queue.add(s)
    while (queue.peek() != null) {
        int v = queue.pop();
        visited[v] = true;

        for (int u: graph[v]) {
            if (!visited[u]) {
                queue.add(u);
            }
        }
    }
}

```

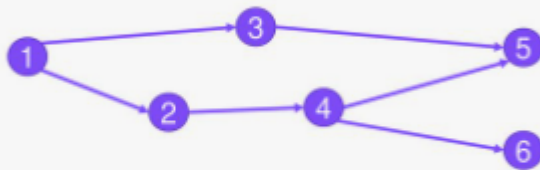
Topsort

Рассмотрим ориентированный граф, присвоим каждой вершине номер.

def Перестановка называется *топологической сортировкой*, если $(,)$ $() < ()$

Пример топологической сортировки

Исходный DAG



Возможные результаты топологической сортировки:

1 – 2 – 4 – 6 – 3 – 5

1 – 3 – 2 – 4 – 5 – 6

1 – 3 – 2 – 4 – 6 – 5

Топологическая сортировка существует только в ациклическом орграфе (т.к. отношение "<" ациклично).

Алгоритм построения:

1. Запустим DFS на всём графе. В момент выхода DFS из вершины будем добавлять её в конец массива.
2. Развернём полученный массив. Он содержит перестановку, являющуюся корректным топсортом:
В DFS, если вершина достигается из v , то v будет обработана раньше (раньше выйдет из стека), поэтому сортировка вершин в порядке убывания времён выхода является топологической сортировкой.

9. Задача поиска кратчайших путей в графе. Алгоритм Дейкстры. Алгоритм Форда–Беллмана.

def Взвешенным графом будем называть тройку (V, E, w) , где V и E – множества вершин и рёбер соответственно, а w – весовая функция.

def Весом пути $v_1 \dots v_n$ будем называть величину $w(v_1, v_2) + w(v_2, v_3) + \dots + w(v_{n-1}, v_n)$

def Кратчайшим путём от вершины s до вершины t , назовём такой путь $s = v_1 \dots v_n = t$, что его вес минимален среди всех возможных путей от s до t . Длину такого пути будем обозначать $dist(s, t)$ и называть расстоянием.

Алгоритм Дейкстры

Цель: Найти расстояния (+ кратчайшие пути) от вершины s до всех остальных, если s – источник.

Алгоритм:

- Пусть S – множество вершин, для которых расстояние от s вычислено корректно на текущий момент времени, $d[v]$ – оценка сверху на $ist(s, v)$ (инициализировать как $+\infty$, $d[s] = 0$).
- Пока $S \neq V$:
 1. Выбрали $u \in V \setminus S$ с минимальным $d[u]$
 2. u присвоим $ist(s, u)$ $d[u]$
 3. Для рёбер вида (u, v) проведём релаксацию, т.е. $d[v] = \min(ist(s, u) + (u, v), d[v])$

Корректность:

st На момент добавления v в S верно, что $ist(s, v) = d[v]$

□

Индукция по размеру :

- **База:** $S = \{s\}$ – всё корректно
- **Переход:** Пусть $v \in V \setminus S$ и $d[v] = \min_{u \in S} (ist(s, u) + (u, v))$. Пусть u – вершина, т.ч. на ней посчитали $d[u]$. Так как из S была релаксация, значит $d[u] = ist(s, u)$. По предположению индукции, $d[u] = ist(s, u)$. Если данный путь не кратчайший, то есть другой устроенный одним из двух образов:
(одно ребро) – путь до u состоит только из вершин из S . Но мы выбрали вершину таким образом, что она заканчивает путь из вершин из S , т.е. этот случай уже рассмотрен.

P – путь начался в s , далее попал в u и далее пришёл в v . Пусть ребро (u, v) такое, что $d[v] = d[u] + (u, v)$. Если таких несколько, то выберем последнее на пути из них. Так как $u \in S$, то $d[u] = ist(s, u)$, при этом известно, что $d[v] \leq ist(s, v)$, т.к. иначе мы бы рассматривали v , а не u . Известно, что $(ist(s, u) + (u, v)) \leq ist(s, v)$, откуда:

$$ist(s, v) \leq ist(s, u) + (u, v) = d[u] + (u, v) = d[v] \leq ist(s, v)$$

Противоречие завершает доказательство.

Время:

- Уменьшение оценки d для вершины: каждое ребро уменьшает оценку не более одного раза, значит таких операций $O(E)$.
- Получение вершины с минимальной оценкой d не из S : каждая вершина извлекается не более одного раза, значит таких операций $O(V)$.

Контейнер вершин	Релаксация	Извлечение	Итого
Массив	$O(1)$	$O()$	$O(2)$
Дерево	$O(\log)$	$O(\log)$	$O(\log)$

(Пути можно восстановить, если куда-то писать, какая вершина вызвала релаксацию в данной, тогда `parent[to] = from`)

Алгоритм Форда-Беллмана

Цель: Найти расстояния от вершины s до всех остальных, если , в предположении, что нет циклов отрицательного веса.

Алгоритм (дп):

1. **Что хранит в себе состояние динамики?** Пусть $dp[s][i]$ равно минимальному весу пути из s в i ровно из i рёбер.
2. **База:** $dp[s][0] = 0$, $dp[s][i] = \infty$
3. **Переход:** $dp[v][i] = \min_u (dp[u][i-1] + w(u,v))$
4. **Какой порядок пересчёта?** Внешний цикл по i , внутри цикл по рёбрам
5. **Где лежит ответ на задачу?** $ans = \min_i dp[s][i]$

Алгоритм рассчитывает матрицу dp . По первому измерению – все вершины, т.е. n , по второму – длина пути, т.е. i от 0 до $n-1$, откуда памяти $O(n^2)$

Время работы вытекает из порядка пересчёта, т.е. $O(n^3)$

def Циклом отрицательного веса назовём цикл $v_1 \dots v_n$, у которого $w(v_1, v_2) + \dots + w(v_n, v_1) < 0$. При поиске кратчайших путей можно ходить по нему бесконечно долго, получая путь сколь угодно малого веса.

Для поиска циклов отрицательного веса можно адаптировать алгоритм Форда-Беллмана: храним для каждой вершины предка, из которого она релаксировалась.

st На i -ой итерации найдена вершина v , до которой расстояние уменьшилось по сравнению с $(i-1)$ -ой итерацией. В графе есть цикл отрицательного веса, достижимый из s .

□

() Простой кратчайший путь не может быть длиннее $n-1$ рёбер, а если произошла релаксация, то существует **не** простой путь, имеющий вес строго меньший, а значит есть цикл отрицательного веса.

() Рассмотрим цикл отрицательного веса $w_1 \dots w_k < 0$, на i -ой итерации найдётся вершина v_i из C , которая будет рассмотрена ещё (второй или более) раз, при этом она будет рассмотрена по пути вдоль отрицательного цикла, а значит произойдёт релаксация.

10. Задача построения минимального остовного дерева. Алгоритм Крускала. Алгоритм Прима.

def Остовом графа (V, E) будет называть граф (V, E') , где

def Остовным деревом графа (V, E) будем называть остов, образующий дерево.

def Минимальным остовным деревом графа (V, E) будем называть такое остовное дерево (V, E') , что E' минимальна. Будем называть его *Minimum Spanning Tree (MST)*.

Лемма о безопасном ребре

def (S, T) – разрез, если $S \cap T = \emptyset$. Ребро (u, v) пересекает разрез, если $u \in S$ и $v \in T$ в разных частях разреза.

def Пусть (V, E') – подграф некоторого MST графа (V, E) . Ребро (u, v) безопасное, если при добавлении его в E' , $(V, E' \cup \{(u, v)\})$ также является подграфом некоторого MST графа (V, E) .

lemm Рассмотрим связный взвешенный граф (V, E) . Пусть (V, E') – подграф некоторого MST графа (V, E) , (S, T) – разрез, т.ч. ни одно ребро из E' не пересекает разрез, а (u, v) – ребро минимального веса, пересекающее разрез (S, T) . Тогда ребро (u, v) является безопасным.

□

Достроим E' до некоторого MST, который обозначим T_{min} . Если $T_{min} = E'$, то лемма доказана, поэтому предположим, что $T_{min} \neq E'$.

Рассмотрим путь в T_{min} от вершины u до вершины v . Хотя бы одно ребро на этом пути пересекает разрез (выберем любое). По условию, $w(u, v) \leq w(e)$. Заменяем ребро e в T_{min} на (u, v) . Полученное дерево также является MST, поскольку все вершины по-прежнему связны и вес дерева не увеличился. (u, v) можно дополнить до миностова в графе (V, E) , т.е. ребро (u, v) – безопасное.

Алгоритм Прима

Цель: Найти MST графа (V, E)

Алгоритм:

Изначально s , где s – произвольная вершина.

1. Рассмотрим разрез $<, >$. Найдём безопасное для него ребро $(,)$, где $.$
2. Добавим $в$ миностов и $в$.
3. Добавим в множество рёбер, пересекающих разрез, рёбра, выходящие из $и$ идущие не в $.$
4. Повторять пока $.$

Сложность:

- Добавление рёбер в множество пересекающих разрез $раз.$
- Удаление ребра минимального веса через разрез $– 1 раз.$

Контейнер Рёбер	Добавление	Извлечение	Итого
Массив	$O(1)$	$O()$	$O(2)$
Дерево Поиска	$O(\log)$	$O(\log)$	$O(+ \log)$

Добавление в массиве можно реализовать так: изначально инициализировать вес равный $+INF$, а далее, при пересечении разреза, ставить реальный вес.

Алгоритм Крускала

Цель: Найти MST графа $(, ,)$

Алгоритм:

1. Отсортируем рёбра по весу.
2. Итерируясь по рёбрам, проверяем, приводит ли его добавление к циклу. Если нет – берём, иначе пропускаем.

Второй шаг легко проверить, используя для хранения компонент связности DSU (Disjoint Set Union) – структуру данных с интерфейсом: $unite(a, b)$ – объединить два множества, где находятся a и b , $are_same(a, b)$ – узнать, лежат ли a и b в одном множестве. В реализации на массиве с весовой эвристикой и эвристикой сжатия путей время на запрос составляет $O((n))$, где n – число вершин, $() min(,)$ – обратная функция Аккермана.

В большинстве случаев $(n) \leq$, однако опускать её при анализе нельзя.

Пример: $(,) 2^{2^{2^{636}}} – 3$

Сама функция Аккермана:

$$A(m, n) = n + 1, \quad m = 0(m - 1, 1), \quad n = 0(m - 1, (m, n - 1)), \quad 1$$

По итогу время работы первого шага – $O(\log)$, второго – $O((\log))$. Итого, время работы алгоритма $O(\log)$.

11. Задача о максимальном потоке в графе. Теорема Форда–Фалкерсона. Алгоритм Эдмондса–Карпа.

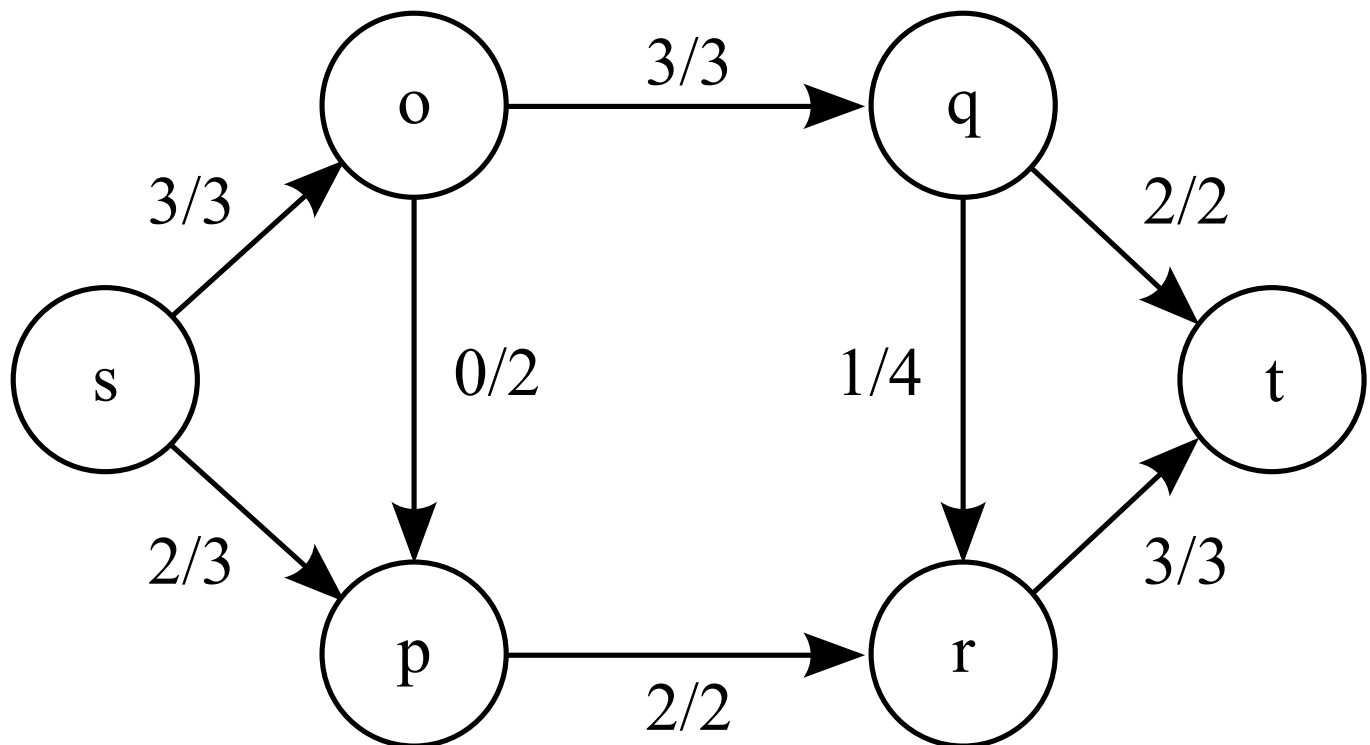
Пару рёбер $(u, v), (v, u)$ в графе G называют антипараллельными.

def Сеть назовём кортеж (G, s, t) , где G – ориентированный граф без антипараллельных рёбер, c – вместимость (capacity), s, t – исток и сток соответственно.

(Далее считаем, что $c(u, v) \geq 0$). Тогда справедливо определять capacity на G)

def Поток в транспортной сети – функция $f: E \rightarrow \mathbb{R}^+$, т.ч.:

1. $0 \leq f(u, v) \leq c(u, v)$ (свойство ограниченности потока)
2. $\sum_{v \in V} f(u, v) = \sum_{v \in V} f(v, u)$ (свойство сохранения потока)



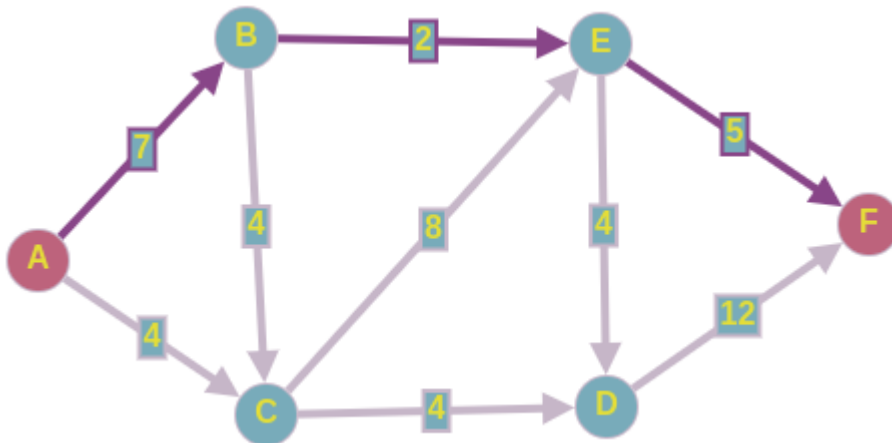
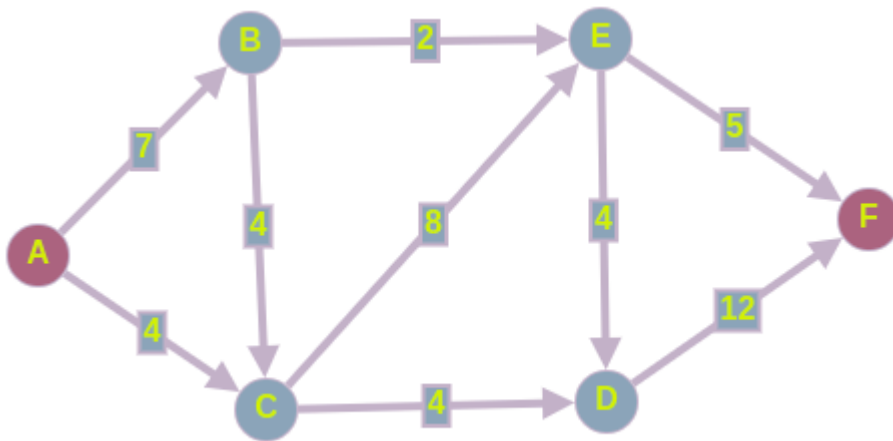
def Пусть в сети зафиксирован поток f . Тогда *величиной потока* называют $f(s, t) - f(t, s)$

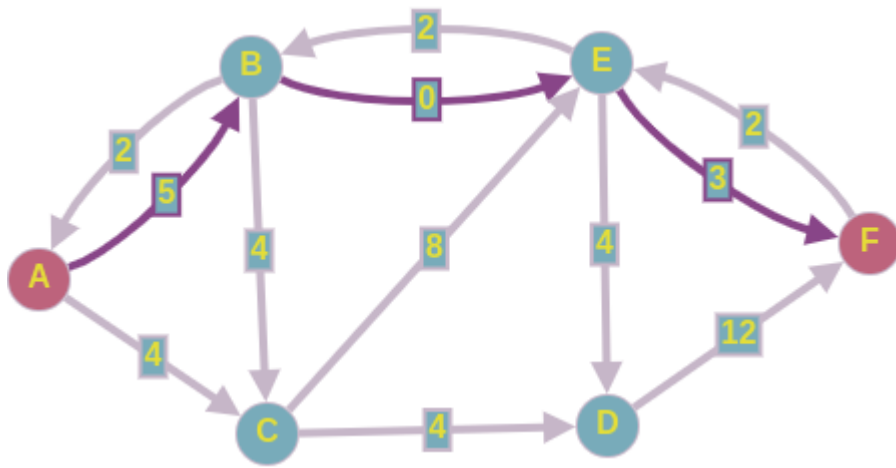
Задача о максимальном потоке – задача поиска в сети такого потока f , что f максимален.

Ещё пара полезных терминов:

def *Остаточная сеть* для сети (V, E, c, f) и потока f – сеть (V, E_f, c_f) , где:

$$E_f = \{ (u, v) \in E \mid 0 < f(u, v) < c(u, v) \} \cup \{ (v, u) \in E \mid 0 < f(u, v) < c(u, v) \}$$





def Дополняющим потоком для сети и потока f определим поток f в остаточной сети f .

def Сложение потоков $f + f$ определяется следующим образом:

$$, (f + f)(,) f(,) + f(,) - f(,), (,) 0, 1$$

st Сложение потока f с дополняющим потоком f также является потоком.

Теорема Форда-Фалкерсона

def Разрезом в сети $(, , s, t)$ назовём пару $(,)$, т.ч.:

- ,
- s, t
- и

def Потоком через разрез $(,)$ назовём величину $f(,) - f(,)$

def Вместимостью или пропускной способностью разреза назовём: $(,) (,)$

def Минимальным разрезом назовём разрез с минимальной пропускной способностью.

st Пусть в сети задан поток f , тогда $(,)$ – разрез $f(,) f$

□

По определению потока $s, t f(,) - f(,) 0$

Тогда, используя определение величины потока:

$$f f(s,) - f(, s) f(s,) - f(, s) +_s (f(,) - f(,))$$

$$f(,) - f(,) f(,) + f(,) - f(,) - f(,)$$

$$f(,) - f(,) f(,)$$

conc Величина любого потока ограничена в сети: не превосходит пропускной способности произвольного разреза.

□

$$f(f(u,v), f(u,v) - f(u,v)) \leq f(u,v) \leq c(u,v)$$

th (Форда-Фалкерсона) Пусть задана сеть и поток f в ней, тогда следующие утверждения эквивалентны:

1. Поток f в сети максимален
2. В f нет пути из истока в сток.
3. Существует разрез (S, T) т.ч. $f(u,v) = c(u,v)$

□

(1 2) Предположим противное: в f есть путь из истока в сток, пустим вдоль него дополняющий поток $f, f > 0$. Тогда $f + f$ – поток и $f + f > f$. Противоречие с максимальнойностью f .

(2 3) Пусть в f нет пути из s в t , тогда определим разрез (S, T) следующим образом: пусть S – множество вершин, достижимых из s в остаточной сети. Рассмотрим пару вершин u, v :

- Если $(u,v) \in E$, то $f(u,v) < c(u,v)$, т.к. иначе $f(u,v) > 0$ и есть в остаточной сети ребро (u,v) , а значит t достижима из s , что неверно.
- Если $(u,v) \in E$, то $f(u,v) = c(u,v)$. Иначе был бы путь из s в t по появившемуся в остаточной сети обратному ребру.

$$\text{Откуда: } f(u,v) = c(u,v) - f(u,v) \leq c(u,v)$$

(3 1) Для любого потока и разреза $f \leq c$, но построен такой поток f , что $f \leq c$ – максимальный.

Алгоритм Эдмондса-Карпа

Цель: Найти максимальный поток в сети.

Алгоритм:

1. Построим сеть G_f , определим поток $f = 0$. Строим остаточную сеть G_f .
2. Пока есть путь из s в t в G_f :
 - Пропустим поток f вдоль него: запускаем BFS из s в t в G_f и пропускаем вдоль пути сразу весь доступный поток (минимальный capacity на пути).
 - Складываем поток $f = f + f$

- Перестраиваем вдоль остаточную сеть.

Анализ времени работы:

Лемма (о неубывании расстояний) Если в сети (G, s, t) увеличение потока производится вдоль кратчайших (рёберная длина) путей из истока в сток в f , то $d_f(s, t)$ длина кратчайшего пути $d_f(s, t)$ в f не убывает.

□

Пусть f, f' – потоки в G , между которыми одна итерация алгоритма. Пусть v – ближайшая вершина, для которой расстояние уменьшилось, т.е. $d_{f'}(s, v) < d_f(s, v)$.

Рассмотрим путь P из s до v , являющийся кратчайшим от s до v в f' . Тогда верно, что $d_f(s, v) \leq d_{f'}(s, v) - 1$. Откуда, из выбора v , следует, что $d_f(s, v) \leq d_{f'}(s, v)$, $d_f(s, v) < d_{f'}(s, v)$ и $d_f(s, v) = d_{f'}(s, v) - 1$.

- Если $(u, v) \in E$, то $d_f(s, v) \leq d_f(s, u) + 1 \leq d_{f'}(s, u) + 1 = d_{f'}(s, v)$
- Если $(u, v) \in E$, но $(u, v) \notin P$, то появление (u, v) означает увеличение потока по обратному ребру (v, u) . Увеличение потока производится вдоль кратчайшего пути, поэтому кратчайший путь из s в v , вдоль которого происходило увеличение, выглядит как $s \rightarrow \dots \rightarrow u \rightarrow v$, откуда:
 $d_f(s, v) = d_f(s, u) + 1 \leq d_{f'}(s, u) + 1 = d_{f'}(s, v) - 2$

Теорема Время работы алгоритма Эдмондса-Карпа составляет $O(E^2)$.

□

Назовём ребро (u, v) вдоль кратчайшего пути в f от s до t , критическим, если $d_f(s, v) = d_f(s, u) + 1$. Покажем, что каждое ребро становится критическим $O(E)$ раз.

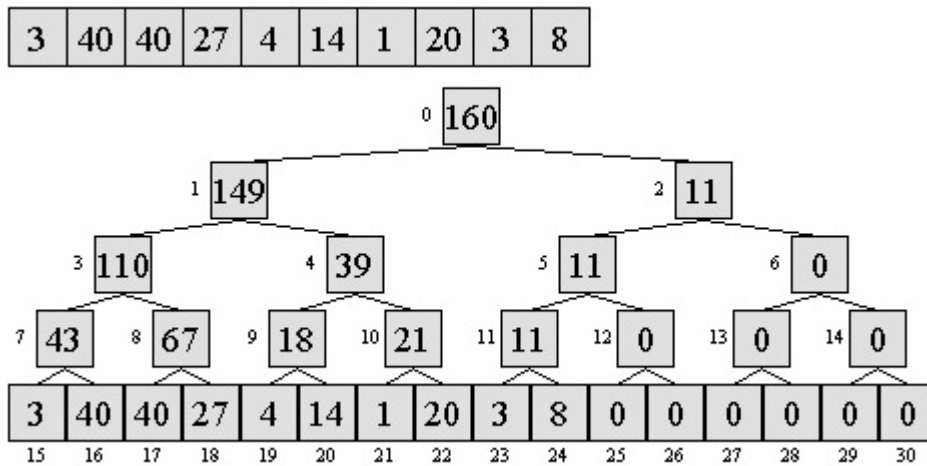
Заметим, что после увеличения все критические рёбра исчезают из остаточной сети. Рассмотрим (u, v) . Увеличение производится вдоль кратчайших путей, поэтому если (u, v) становится критическим в первый раз, то $d_f(s, v) = d_f(s, u) + 1$. Затем оно исчезнет из сети, пока поток по нему не будет уменьшен по обратному ребру (v, u) . Пусть в момент перед увеличением поток в сети G составлял f , тогда $d_f(s, v) = d_f(s, u) + 1$. Из леммы $d_f(s, u) \leq d_{f'}(s, u)$, откуда: $d_{f'}(s, v) = d_{f'}(s, u) + 1 \leq d_f(s, u) + 1 = d_f(s, v) + 1$. Получаем, что между итерациями, когда (u, v) становится критическим, расстояние от s до v увеличивается на 2 ребра, откуда $O(E)$ раз оно могло становиться критическим.

Всего рёбер $O(E)$, значит суммарное число итераций составит $O(E^2)$. Время работы каждой итерации – $O(E)$. Тогда итоговое время работы алгоритма Эдмондса-Карпа составит $O(E^3)$.

12. Дерево отрезков. Ограничения на поддерживаемую операцию. Запросы модификации на отрезке.

def *Дерево отрезков* способно за $O(\lg N)$ получать на подотрезке результат любой операции, которая ассоциативна, коммутативна и имеет нейтральный элемент.

В частности, задачи RSQ/RMQ, как правило, решаются с использованием этой структуры данных.



- **Построение:**

Опишем нерекурсивное построение дерева на массиве длины N . Считаем, что $\log_2 N$, иначе дозаполним до степени двойки нейтральным элементом.

Заведём массив длины $2N - 1$, последние N элементов будут элементами исходного массива. Первые $N - 1$ элементов заполним следующим образом:

$t[i]$ ($t[2i + 1], t[2i + 2]$) (заполнение от элемента с номером $N - 2$ и до 0 в цикле).

- **Запрос по отрезку:**

Пусть мы находимся в узле, отвечающего за подотрезок $[l, r]$, а изначальный запрос был $[,]$.

- Если $[l, r] [,]$, то вернем нейтральный элемент.
- Если $[l, r] [,]$, то вернём ответ в узле.
- Иначе вернём результат операции с ответов детей.

То есть исходный отрезок разбивается на дизъюнктное объединение подотрезков.

Заметим, что на каждом уровне раскрываться вниз могут не более двух узлов, так как только крайние могут порождать дочерние запросы. Следовательно, время работы $O(\lg N)$.

- **Модификация в точке:**

Пусть необходимо обновить элемент с индексом i .

1. Найдем в дереве лист, отвечающий за i -й элемент. Это $(N - 1 + i)$ -й.
2. Индекс родителя: $i - 1$
3. Обновляем значение в родителе через результаты на деревьях.
4. Повтори, пока не обновил корень.

Запросы модификации на отрезке

def Операцией группового обновления будем называть запрос, в ходе которого применяется на подотрезке операция $(\cdot, al)$.

Свойства на операцию $\cdot$, если $\cdot$ – операция запроса на отрезке:

1. Существование нейтрального элемента.
2. Ассоциативность: $(a, (b, \cdot)) = ((a, b), \cdot)$
3. Дистрибутивность: $((a, b), \cdot) = ((a, \cdot), (b, \cdot))$

def Несогласованность – величина, с которой надо выполнить операцию $\cdot$, чтобы получить корректный результат.

В каждый момент времени в узле необязательно лежит истинное значение на отрезке. Но к моменту запроса будем осуществлять *проталкивание несогласованности*.

Псевдокод из [презы](#):

```
void push(int node) {
    tree[2*node + 1].d = ch(tree[2*node + 1].d, tree[node].d);
    tree[2*node + 2].d = ch(tree[2*node + 2].d, tree[node].d);
    tree[node].d = ch_neutral;
}
```

Проталкивание несогласованности детям. Необходимо выполнять как только идет рекурсивный запуск от текущей вершины к её детям. Нужно это для того, чтобы в детях в момент обработки были корректные данные.

```
void update(int node, int a, int b, T val) {
    l = tree[node].left, r = tree[node].right;
    if ([l, r] is subset [a, b]):
        tree[node].d = ch(tree[node].d, val);
        return;
    push(node);
    update(2 * node + 1, a, b, val);
    update(2 * node + 2, a, b, val);
}
```

```

    tree[node].ans =
        op(ch(tree[2*node+1].ans, tree[2*node+1].d),
           ch(tree[2*node+2].ans, tree[2*node+2].d));
}

```

Процедура обновления на отрезке. Данная процедура выполняет разбиение текущего отрезка на подотрезки и обновление в них несогласованности. Очень важно выполнить `push` как только идет рекурсивный вызов от детей, чтобы избежать некорректной обработки в детях. И так как значение в детях могло измениться, то необходимо выполнить обновление ответа по операции на текущем отрезке.

```

...
ans = op(query(2*node + 1, a, b), query(2*node + 2, a, b));
tree[node].ans = op(ch(tree[2*node + 1].ans, tree[2*node + 1].d),
                   ch(tree[2*node + 2].ans, tree[2*node + 2].d));
return ans;

```

Получение ответа по операции отличается от операции обновления лишь в том, что для каждого отрезка разбиения необходимо не обновить несогласованность, а сложить по операции с текущим ответом истинное значение на отрезке (то есть результат сложения по операции значения в вершине с несогласованностью).

13. Быстрое преобразование Фурье. Применение для умножения длинных чисел и многочленов.

Алгоритм FFT

Рассмотрим задачу интерполяции $\cdot a$ y , где известны значения многочлена

$(\cdot) a_0 + a_1 + \dots + a_{n-1} x^{n-1}$ в комплексных корнях из единицы степени n , т.е. $\omega_n^{2in}, 0, 1, \dots, n-1$.

Тогда в матричном виде задачи будет фигурировать матрица:

$$\begin{pmatrix}
 0 & 0 & n & 0 & 1 & n & \dots & 0 & (n-1) & n & 1 & 0 & n & 1 & 1 & n & \dots & 1 & (n-1) & n & \dots & (n-1) & 0 & n & (n-1) & 1 & n & \dots & (n-1) & (n-1) & n
 \end{pmatrix}$$

Для решения задачи интерполяции надо найти $^{-1}$ (она существует, можно проверить с помощью теоремы Вандермонда).

$$st^{-1} \quad 1n, ij \quad -ij, j \quad 0, \dots, n-1$$

□

$$()_{ij} \quad n-1 \quad 0 \quad n-1 \quad 0 \quad (i-j) \quad n-1 \quad 0 \quad n, \quad i \quad j \quad i-j(1-n(i-j))1-(i-j) \quad i-j(1-1)1-(i-j) \quad 0, \quad i \quad j$$

def Пусть задан многочлен $() \quad a_0 + a_1 + \dots + a_n^n$, тогда *дискретным преобразованием Фурье* называется вектор $() \quad ((0n), (1n), \dots, (n-1n))$

Преобразование Фурье можно записать в матричном виде как $\cdot a \quad y$, где a – вектор коэффициентов, а y – вектор, равный $()$.

Таким образом, надо научиться быстро умножать матрицу на вектор.

Алгоритм:

1. Дополним многочлен нулевыми коэффициентами до степени двойки.
2. Разбиваем вычисление $()$ на рекурсивное вычисление для $0 \quad a_0 + a_2 + a^2 + \dots$ и $1 \quad a_1 + a_3 + a^2 + \dots$
3. С помощью преобразования бабочки восстанавливаем $()$.

Пусть $(0) \quad (b_0, \dots, b_{n2-1})$ и $(1) \quad (c_0, \dots, c_{n2-1})$.

Первые $n2$ значений: $() \quad b + n$

Вторая половина, т.е. $n2 + :$

$$()_{n2+} \quad (n2+) \quad 0(n2+) + n2+ \cdot 1(n2+) \quad 0(2) + n2_1(2) \quad b - n2 - 1 \text{ т.к. } n - \text{степень двойки.}$$

Время работы всего алгоритма определяется рекуррентой $(n) \quad 2(n2) + O(n)$, откуда время вычисления составит $O(n \log n), n$

Умножение многочленов

Пусть $() \quad a_0 + a_1 + \dots + a_{n-1}^{n-1}$, $() \quad b_0 + b_1 + \dots + b_{m-1}^{m-1}$. Нужно их перемножить за $O((n+m) \log(n+m))$.

Пусть $2^{\log_2 n+m+1}$, дополним i нулями до такой длины.

1. Посчитаем $()$ и $()$
2. Посчитаем $() \quad () \cdot ()$
3. Вычислим обратное преобразование Фурье, т.е. по $()$ найдём :

$() \quad a$, где a – вектор коэффициентов, а $(ij)_{n-1i,j0}$. С помощью FFT умеем делать это за $O(\log)$. В матричной постановке обратное преобразование имеет вид $^{-1} \cdot ()$. Уже

знаем что $a^{-1} \equiv 1/n$, где $(-ij)_{n-1i,j0}$. То есть нужно посчитать всё то же самое, но с a^{-1} вместо a , и поделить на n в конце.

Для умножения длинных чисел достаточно представить их как многочлены ($x=10$), перемножить с помощью FFT, а потом восстановить значения.

14. Нахождение обратного элемента по модулю. Быстрое возведение в степень.

Обратный элемент в m

def $a^{-1} \equiv b$ в m , если $a \cdot b \equiv 1$

st 1) Если m – простое, то $a^{-1} \equiv a^{m-2}$ в m . (Следствие из МТФ); 2) Пусть m , $a < m$. Тогда $a^{-1} \equiv (a, m)^{-1}$ и $a^{-1} \equiv a^{(m)-1}$ (Следствие из теоремы Эйлера).

Алгоритм Евклида (псевдокод):

```
int gcd(int a, int b) {
    while ( b != 0 ) {
        a %= b;
        swap(a, b);
    }
    return a;
}

int GCD(int a, int b) {
    return b ? GCD(b, a % b) : a;
}
```

Расширенный алгоритм Евклида:

Рассмотрим уравнение $ax + by = (a, b)$. Пусть было найдено решение для (b, a) , перейдём к (a, b) :

$$ba \equiv b - ba \cdot a \equiv (b - ba \cdot a)_1 + ay_1 \equiv (a, b) \equiv a(y_1 - ba \cdot 1)b_1 \equiv (a, b)$$

Следовательно:

$$y_1 - ba \cdot 1 \equiv y_1$$

Тогда обратный элемент можно найти следующим способом:

Рассмотрим систему $ax + my = 1$. Возьмём обе части по модулю m : $a \equiv 1 \equiv a^{-1}$

То есть с помощью расширенного алгоритма Евклида ищем обратный элемент в m за $O(\log m)$.

Быстрое возведение в степень.

def Алгоритм быстрого возведения в степень – алгоритм, предназначенный для возведения числа в натуральную степень n за меньшее число умножений, чем это требуется в определении.

Нетрудно заметить такое соотношение:

$$a^n = \begin{cases} a^{(n/2)^2}, & n \text{ нечётное} \\ a^{(n/2)^2}, & n \text{ чётное} \end{cases}$$

Тогда в простейшем виде алгоритм можно реализовать так:

```
def my_power(val, p):
    if p < 0:
        return my_power(1/val, -p)
    if p == 0:
        return 1
    if p % 2 == 0:
        return my_power(val*val, p/2)
    else:
        return val*my_power(val*val, (p-1)/2)
```

15. Задача поиска подстроки в строке. Алгоритм Рабина–Карпа. Алгоритм Кнута–Морриса–Пратта.

Задача: Дан текст и паттерн. Требуется найти все позиции, начиная с которых входит в.

Определим префикс-функцию и линейный алгоритм её подсчёта, чтобы использовать в алгоритме Кнута-Морриса-Пратта. Аналогично можно посчитать с z-функцией.

def Строка называется *супрефиксом*, если она является одновременно и префиксом, и суффиксом строки.

def Префикс-функцией от строки называют массив (f_i) длины n , что f_i равно длине максимального несобственного (не равного всей строке) супрефикса строки $[i+1]$.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
a	a	b	#	b	a	a	b	c	a	b	a	a	b	a	a	b	a	b
0	1	0	0	0	1	2	3	0	1	0	1	2	3	1	2	3	1	0

$st()[i+1] - st()[i] + 1$

□

Рассмотрим подстроку вида: суффикс строки $[i+2]$ длины $st()[i+1]$. Удалив последний символ этой подстроки мы получим суффикс, оканчивающийся на позиции i и имеющий длину $st()[i+1] - 1 = st()[i]$

st Супрефиксы в порядке вложенности имеют длины $st()[0]$, $st()[1]$, $st()[2]$, ...

□

Рассмотрим супрефикс строки длины $st()[0]$, по определению – это максимальный супрефикс. Следующий по вложенности супрефикс – супрефикс подстроки $[1]$.

Линейный алгоритм: Будем насчитывать индукцией по i :

- База: $st()[0] = 0$, по определению.
- Подсчёт $st()[i+1]$: пусть $j < i$ известно $st()[j]$. Надо посчитать $st()[i+1]$. Рассмотрим два случая:
 - Если $[i+1] \in [j]$, то $st()[i+1] = st()[j] + 1$ по определению.
 - Если $[i+1] \notin [j]$. Цель – максимальный по длине супрефикс, заканчивающийся в $(i+1)$ -ой позиции. Он состоит из какого-то супрефикса строки $[i+1]$ и символа $[i+1]$. Переберём все супрефиксы $[i+1]$ в порядке вложенности, пусть они имеют длины $j_1 > j_2 > \dots > j_m$, тогда $st()[i+1] = j_m + 1$, где m – первый из последовательности, такой что $[j_m + 1] \in [i+1]$.

Объединяя два случая выше и пользуясь утверждениями: $st()[i+1] = st()[j] + 1$, где j – такой индекс, что $[j] \in [i+1]$ и при этом j – первый такой индекс из последовательности $i, i-1, i-2, \dots$

Псевдокод:

```
def prefixFunction(s: str):
    p = [0] * len(s)
    for i in range(1, len(s)):
        k = p[i - 1]
        while k > 0 and s[i] != s[k]:
            k = p[k]
```



```

        k = p[k - 1]
        if s[i] == s[k]
            k++
        p[i] = k
    return p

```

Время работы: определяется числом итераций внутреннего цикла `while`: счётчик `k` может увеличиться не больше, чем на 1, значит оно не больше -1 . А внутри `while` оно убывает, значит суммарное число итераций `while` не превосходит -1 (ограниченно сверху числом увеличений счётчика). Итоговое время работы составит $O()$

Алгоритм Кнута-Морриса-Пратта

Цель: Дан текст `t` и паттерн `p`, найти все вхождения паттерна в текст.

Алгоритм:

- Построим строку `lps`, где `lps[i]` — любой символ, не входящий в алфавит `t` и `p`.
- Вычислим `lps`
- Если в какой-то позиции `t[i]`, значит мы нашли конец очередного вхождения `p` в `t`, значит начало вхождения — в позиции `i - lps[i] + 1`.

Алгоритм Рабина-Карпа

def Полиномиальная хеш-функция для строки `s` определяется, как
$$h(s) = \sum_{i=0}^{l-1} s[i] \cdot a^i \pmod{m}$$
, где a — простое, а m (мультипликативного кольца).

Заметим, что
$$h(s) = \sum_{i=0}^{l-1} s[i] \cdot a^i \pmod{m} + \sum_{i=0}^{l-1} s[i+l] \cdot a^i \pmod{m} = (h(s) + a^l \cdot h(s)) \pmod{m}$$

st Пусть $l = |p|$, тогда $h(p) = \sum_{i=0}^{l-1} p[i] \cdot a^i \pmod{m}$

□

$$h(p) = \sum_{i=0}^{l-1} p[i] \cdot a^i \pmod{m} = \sum_{i=0}^{l-1} p[i] \cdot a^i \pmod{m} = \sum_{i=0}^{l-1} p[i] \cdot a^i \pmod{m}$$

Эта вся нужная информация, чтобы говорить об алгоритме.

Цель: Поиск паттерна `p` в тексте `t`

Алгоритм:

1. Посчитаем полиномиальный хеш паттерна за $O()$
2. Посчитаем массив префиксных хешей за $O()$
3. Переберём в тексте все подстроки размера `l` и для каждой сравним хеш с хешом `p`. Если не совпали, то точно пропускаем, если совпали, то проверяем в лоб.

Как следствие, путём выбора простого $q > 2$ получаем, что вероятность коллизии меньше $\frac{1}{q}$, а значит в среднем время работы составит $O(n + (1 + \frac{1}{q})) = O(n)$

- *Поиск строки:* Идём по бору, если в какой-то момент нет ребра по символу или закончили не в терминале, то строки нет в исходном множестве.
- *Добавление строки:* Идём по бору. Если в какой-то момент не смогли пойти – дописываем, создавая новую ветку. Конечная вершина должна стать терминалом.
- *Удаление строки:* Находим строку, далее удаляем её с конца, пока не наткнёмся на терминальную вершину или на вершину с более чем одним ребёнком.

Контейнер	Построение	Поиск	Память
Массив	$O(it)$	$O()$	$O(it)$
map (на дереве поиска)	$O(\log_{it})$	$O(\log)$	$O(it)$
hash_map	$O(it)$	$O()$	$O(it)$

где σ – алфавит, it – слово.

Алгоритм Ахо-Корасик

Пусть u – слово, которое составляет путь от корня до вершины u в боре.

def Суффиксной ссылкой вершины называют такую вершину $lin(u)$, что u является максимальным по длине суффиксом u , который можно прочесть, идя из корня до, быть может, нетерминальной вершины.

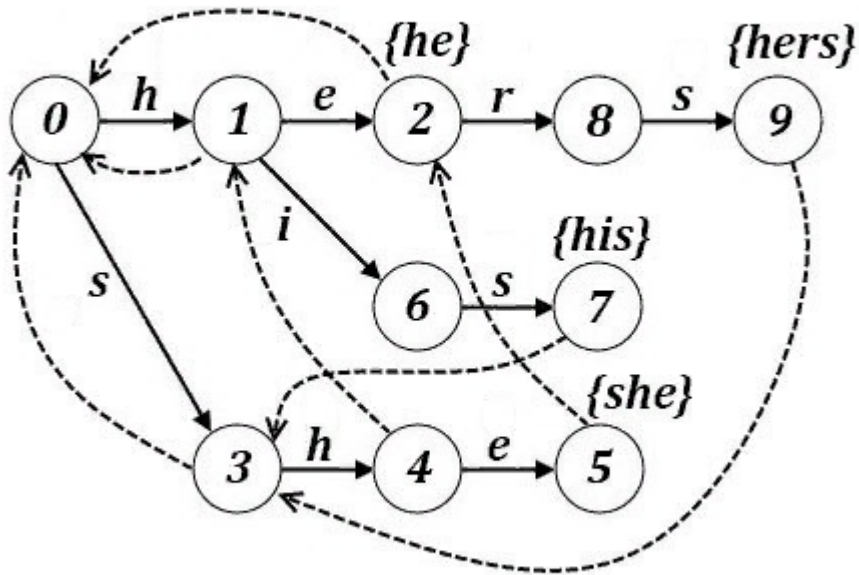
Суффиксная ссылка для корня, в общем случае, не определена. Её можно доопределить как `null`, либо пропускать в коде.

def Введём функцию $t(u, c)$, равную вершине, куда из вершины u можно перейти по букве c . Определяется следующим образом:

$$t(u, c) = \begin{cases} rt(u + c), & \text{если из вершины } u \text{ есть переход по } c \\ lin(u), & \text{если из вершины } u \text{ нет перехода по } c \end{cases}$$

st Если переход из u по букве c есть в боре, то $lin(rt(u + c)) = rt([t(lin(u), c)])$

def Автомат Ахо-Корасик – детерминированный конечный автомат (ДКА), построенный на боре с суффиксными ссылками и функцией перехода $t(\cdot, \cdot)$.



Алгоритм построения (алгоритм Ахо-Корасик):

1. Строим бор на данном наборе слов.
2. С помощью BFS из корня насчитаем функции lin и t пользуясь утверждением выше.

Корректность:

Из соотношения выше следует, что функции можно насчитать через вершины выше текущей, для которых всё верно по предположению индукции. База индукции – корень, для него посчитаем всё по определению.

Поиск множества паттернов в тексте:

Задача: Пусть есть набор паттернов $p_1, \dots, p_k$ и текст t , в котором мы хотим найти все вхождения этих паттернов.

def Сжатой суффиксной ссылкой вершины называют такую вершину $m()$, что

$$m() = \begin{cases} lin(), & \text{если } l() \text{ терминал} \\ m(lin()), & \text{иначе} \end{cases}$$

Алгоритм:

1. Построим автомат Ахо-Корасик за $O(\sum |p_i|)$
2. Теперь будем идти по функции t текстом из корня за $O(n)$
3. При очередном переходе будем прыгать по сжатым суффиксным ссылкам вверх и насчитывать вхождения.

Каждый прыжок по суффиксной ссылке ведёт либо в корень, либо даёт новое вхождение, т.е. прыжков будет $O(ans)$, где ans – суммарное число вхождений всех паттернов в текст.